UNIDAD 2 — MÉTODOS DE BÚSQUEDA

Material complementario del curso de Inteligencia Artificial. Esta unidad cubre los agentes inteligentes, los métodos de búsqueda (en anchura y profundidad), los árboles de decisión, la teoría de juegos y la satisfacción de restricciones.

Temas de la Unidad 2:

- Agentes inteligentes
- Búsquedas (en anchura y en profundidad)
- Árboles de decisión
- Teoría de juegos
- Satisfacción de restricciones

1. Agentes inteligentes

El agente es el concepto fundamental sobre el que se construye toda la Inteligencia Artificial moderna según el libro de Russell y Norvig. No podemos hablar de búsquedas, árboles de decisión o juegos sin antes entender quién es el "sujeto" que realiza todas estas acciones.

Un agente es cualquier cosa que puede ser considerada como percibiendo su entorno a través de sensores y actuando sobre ese entorno a través de actuadores. La clave aquí es el ciclo percepción-acción. Un agente no es solo un programa; es una entidad que interactúa con un mundo, ya sea físico o virtual. Esta definición es tan amplia que cubre desde un termostato hasta un robot humanoide, pasando por un programa de ajedrez o un chatbot.

1.1 El diagrama fundamental: Agente = Arquitectura + Programa

- **Arquitectura:** es la "máquina" física o virtual donde se ejecuta el programa. Puede ser un computador con cámaras y ruedas (un robot), o simplemente el hardware de un servidor (un agente de software).
- **Programa del Agente:** es la "mente" del agente. Es la función que implementa la lógica de decisión, mapeando las percepciones a las acciones.

1.2 Componentes clave de un agente

1. Percepción (Sensores): Es la entrada de información del mundo. Los sensores pueden ser:

- Físicos: Cámaras, micrófonos, sensores de tacto, GPS, sonar (para robots).
- Lógicos/Software: Entrada de teclado, archivos leídos, paquetes de red, bases de datos (para softbots o agentes de software como el de compras por Internet).

2. Acción (Actuadores): Es la salida del agente que modifica el entorno. Los actuadores pueden ser:

- Físicos: Motores, ruedas, brazos robóticos, pinzas.
- Lógicos/Software: Escribir en un archivo, enviar un correo electrónico, mostrar un resultado en pantalla, o generar la siguiente palabra en una conversación.

3. Función del Agente vs. Programa del Agente: El libro hace una distinción crítica. La Función del Agente es una descripción matemática abstracta: una tabla ideal que mapea cada posible secuencia de percepciones a una acción. El Programa del Agente es la implementación concreta de esa función en una arquitectura real; es el código que se ejecuta.

1.3 Racionalidad: ¿qué hace que un agente sea bueno o inteligente?

Tener un agente no es suficiente; queremos que sea racional. Un agente racional es aquel que hace "lo correcto" para maximizar su medida de rendimiento. Un agente racional no es omnisciente (no lo sabe todo), sino que actúa basándose en lo que espera que ocurra, dadas sus percepciones y su conocimiento.

Definición formal: Para cada posible secuencia de percepciones, un agente racional debe realizar la acción que se espera que maximice su medida de rendimiento, basándose en la evidencia proporcionada por la secuencia de percepciones y en cualquier conocimiento previo que tenga.

Esto depende de cuatro factores:

- La medida de rendimiento (qué define el éxito).
- El conocimiento previo que tiene el agente sobre el entorno.
- Las acciones que el agente puede realizar.
- La secuencia de percepciones que ha tenido hasta el momento.

1.4 Tipos de Agentes (Estructuras Básicas)

El libro describe una jerarquía de agentes, cada uno más complejo y flexible que el anterior:

Agente Reactivo Simple (Reflexivo): Selecciona acciones basándose solo en la percepción actual. Es un mapeo directo de condición-acción (reglas "si-entonces"). Ejemplo: el agente aspiradora que aspira si ve suciedad, y se mueve si ve que está limpio. Limitación: solo funciona si el entorno es totalmente observable. Si es parcialmente observable, puede caer en bucles infinitos.

Agente Reactivo Basado en Modelos: Mantiene un estado interno que refleja los aspectos del mundo que no puede ver directamente. Este estado se actualiza con las percepciones y las acciones realizadas. Ejemplo: un robot que recuerda que la puerta a su izquierda estaba cerrada en el último fotograma. Ventaja: puede manejar entornos parcialmente observables.

Agente Basado en Objetivos (Metas): Además del estado interno, tiene información sobre una meta (un estado deseado). Elige acciones que lo acerquen a esa meta, a menudo planificando secuencias de acciones. Ejemplo: el agente del mapa de Rumanía que busca una ruta a Bucarest. Ventaja: es más flexible; si el objetivo cambia, el comportamiento se adapta automáticamente.

Agente Basado en Utilidad: Tiene una función de utilidad que asigna un valor numérico a cada estado. Elige la acción que maximiza la utilidad esperada. Ejemplo: el taxista automático que debe equilibrar velocidad, seguridad, comodidad y consumo de combustible. Ventaja: puede manejar objetivos en conflicto y decidir racionalmente bajo incertidumbre.

Agente que Aprende: Tiene un componente de aprendizaje que mejora el rendimiento del agente con la experiencia. Puede aprender nuevos modelos, nuevas políticas o nuevas funciones de utilidad. Ejemplo: TD-Gammon (el jugador de backgammon) que aprendió a jugar mejor jugando millones de partidas contra sí mismo.

2. Métodos de Búsqueda: Anchura y Profundidad

El problema: un agente se encuentra en un estado inicial y debe encontrar una secuencia de acciones que lo lleve a un estado objetivo. El espacio de todos los estados posibles puede ser representado como un grafo. La búsqueda es el proceso de explorar este grafo para encontrar un camino solución.

2.1 Búsqueda en Anchura (BFS - Breadth-First Search)

Esencia: explora el grafo por niveles. Primero expande todos los nodos a una profundidad, luego todos los nodos a la siguiente, y así sucesivamente. Es como una onda expansiva desde el punto de inicio.

Estructura de datos: utiliza una cola (FIFO - First In, First Out). Los nodos que se descubren primero se exploran primero.

Propiedades (según el Capítulo 3 del libro de Russell y Norvig):

- **Completo:** Si existe una solución, BFS la encontrará, siempre que el factor de ramificación sea finito.
- **Óptimo:** Sí, siempre y cuando el costo de cada acción sea el mismo (o el costo del camino sea una función no decreciente de la profundidad).
- **Complejidad temporal:** O(b^(d+1)), donde b es el factor de ramificación y d es la profundidad de la solución más superficial. Es exponencial.
- **Complejidad espacial:** O(b^(d+1)). Este es su principal punto débil. Guarda todos los nodos generados en la frontera, lo que lo hace impracticable para problemas grandes.

Cuándo usarlo: Ideal para problemas donde la solución está cerca de la raíz y el espacio de búsqueda no es demasiado grande (como el mapa de Rumanía para encontrar la ruta más corta entre ciudades). Es una estrategia no informada (a ciegas, fuerza bruta), ya que no utiliza ninguna información adicional sobre el problema.

2.2 Búsqueda en Profundidad (DFS - Depth-First Search)

Esencia: explora un camino hasta el final (una hoja o un callejón sin salida) y luego retrocede (backtracking) para explorar el siguiente camino. Es como moverse por un laberinto sin memoria de los caminos alternativos.

Estructura de datos: utiliza una pila (LIFO - Last In, First Out). Se implementa fácilmente con recursión.

Propiedades:

- **Completo:** No. Puede caer en un bucle infinito si no se tiene cuidado (por ejemplo, un grafo con ciclos). Se puede volver completo usando un límite de profundidad.
- **Óptimo:** No. No garantiza encontrar la ruta más corta; simplemente encuentra una solución.
- **Complejidad temporal:** O(b^m), donde m es la profundidad máxima de cualquier nodo. En el peor caso, explora todo el árbol.

- **Complejidad espacial:** O(bm). Su gran ventaja es que solo necesita almacenar el camino actual y los nodos hermanos no expandidos. Es muy eficiente en memoria.

Variante crucial: Búsqueda de Profundidad Limitada y Búsqueda con Profundidad Iterativa (IDDFS). El libro destaca la IDDFS como una estrategia óptima y completa que combina la eficiencia espacial de DFS con la optimalidad de BFS. Incrementa el límite de profundidad de forma iterativa hasta encontrar una solución.

2.3 Formulación del problema

Un agente basado en objetivos no puede simplemente "buscar" sin una estructura clara. Para implementar cualquier tipo de búsqueda, se necesitan cuatro componentes:

- **Estado Inicial:** ¿Dónde comienza el agente? (Ej: En(Arad) para el problema de Rumanía).
- **Función de Transición (o Sucesor):** Dado un estado y una acción, ¿cuál es el nuevo estado? Define el espacio de estados y el factor de ramificación (b).
- **Test de Objetivo:** ¿Cómo sabe el agente que ha llegado a su meta?
- **Coste del Camino:** ¿Cuánto "cuesta" cada acción? Vital para elegir una solución óptima.

3. Árboles de Decisión

El problema: aprender una función que clasifique ejemplos en base a sus atributos. Es un método de aprendizaje supervisado (Capítulo 18).

3.1 Fundamentos matemáticos

Un árbol de decisión divide el espacio de búsqueda en regiones más pequeñas, realizando preguntas sobre los atributos de los ejemplos. Para decidir qué atributo usar en cada nodo, se utiliza la Ganancia de Información, basada en la Entropía.

Entropía: Mide el desorden o la incertidumbre de un conjunto de datos. Cuanto más mezclados están los ejemplos de diferentes clases, mayor es la entropía. Se calcula como: H(S) = - sumatoria de pi * log2(pi), donde pi es la proporción de ejemplos de la clase i en el conjunto S.

Ganancia de Información (IG): Mide la reducción esperada en la entropía después de dividir los datos según un atributo A. Se calcula como: IG(S, A) = H(S) - sumatoria de ((|Sv| / |S|) * H(Sv)). El algoritmo ID3 (y sus sucesores como C4.5) elige el atributo con la mayor Ganancia de Información para ser la raíz del árbol.

3.2 Overfitting (sobreajuste) y Poda

Problema (Overfitting): Si el árbol crece demasiado, puede "memorizar" los datos de entrenamiento, incluyendo el ruido, y fallar al clasificar nuevos ejemplos.

Solución (Poda): Reducir el tamaño del árbol para mejorar su capacidad de generalización.

- **Pre-poda:** Detener el crecimiento del árbol antes de que se ajuste perfectamente a los datos (por ejemplo, usando un umbral de Ganancia de Información mínimo).
- **Post-poda:** Dejar que el árbol crezca completamente y luego eliminar las ramas que no aportan significativamente a su rendimiento.

3.3 Random Forests

Para mejorar la precisión y reducir el overfitting, se pueden construir múltiples árboles de decisión y combinar sus predicciones, formando un Random Forest. Cada árbol se entrena con una muestra aleatoria de los datos y en cada nodo se considera un subconjunto aleatorio de atributos. La predicción final se obtiene por votación.

4. Teoría de Juegos (IA Adversaria)

El problema: un agente debe tomar decisiones en un entorno donde hay otro agente (el oponente) que tiene objetivos opuestos. Se busca la estrategia óptima, suponiendo que el oponente también juega de forma óptima (Capítulo 6).

4.1 El Algoritmo Minimax

Modela el juego como un árbol donde los niveles alternan entre el Jugador MAX (que quiere maximizar el resultado) y el Jugador MIN (que quiere minimizarlo).

El algoritmo asigna un valor a cada nodo terminal (la utilidad del estado, como +1 para victoria, -1 para derrota).

Propagación: Los valores de los nodos se propagan hacia arriba. En un nodo MAX, el valor es el máximo de los valores de sus hijos. En un nodo MIN, el valor es el mínimo de los valores de sus hijos. La decisión en el nodo raíz es la acción que lleva al hijo con el valor más alto.

4.2 Poda Alfa-Beta

Minimax puro es exponencialmente costoso. La poda alfa-beta mejora la eficiencia sin comprometer la optimalidad. Mantiene dos valores:

- **α (alfa):** el valor de la mejor opción (más alta) que MAX ha encontrado para él.
- **β (beta):** el valor de la mejor opción (más baja) que MIN ha encontrado para él.

Regla de Poda: Si en un nodo MAX se encuentra un valor $v \geq \beta$, se puede podar el resto de hijos. Si en un nodo MIN se encuentra un valor $v \leq \alpha$, se puede podar el resto de hijos.

Con un buen orden de los movimientos, la poda alfa-beta puede reducir la complejidad de O(b^d) a O(b^(d/2)), permitiendo explorar el doble de profundidad.

4.3 Juegos Estocásticos (Expectiminimax)

Para juegos que incluyen azar (como el Backgammon), se añaden nodos de posibilidad. El valor de un nodo de posibilidad es el promedio ponderado de sus hijos: Valor(nodo posibilidad) = sumatoria de (Probabilidad(resultado) * Valor(hijo)).

5. Satisfacción de Restricciones (CSP)

El problema: dado un conjunto de variables (cada una con un dominio de valores posibles) y un conjunto de restricciones entre ellas, encontrar una asignación de valores a las variables que satisfaga todas las restricciones. Se busca una solución factible, no una óptima (Capítulo 5).

5.1 Definición Formal

- **Variables:** X = {X1, X2, ..., Xn}

- **Dominios:** D = {D1, D2, ..., Dn} (conjuntos de valores posibles, usualmente finitos y discretos).
- **Restricciones:** C = {C1, C2, ..., Cm}. Cada restricción involucra un subconjunto de variables y define las combinaciones permitidas.

Ejemplos del libro: el problema de colorear el mapa de Australia (variables: regiones; dominios: colores; restricción: regiones adyacentes no pueden tener el mismo color) y el problema de las 8-reinas.

5.2 Algoritmo de Backtracking con heurísticas

El enfoque estándar para resolver un CSP es la búsqueda con vuelta atrás (backtracking), que es un DFS que asigna valores a las variables de forma incremental. Heurísticas cruciales:

- **MRV (Mínimos Valores Restantes):** "Falla rápido". Elige la variable con el dominio más pequeño (la que tiene menos valores posibles). Reduce drásticamente el factor de ramificación.
- **Grado (Degree Heuristic):** Para desempatar, elige la variable que está involucrada en más restricciones con otras variables no asignadas.
- **LCV (Valor Menos Restringido):** Para ordenar los valores. Elige primero el valor que elimina menos valores de los dominios de las variables vecinas, maximizando la flexibilidad futura.

5.3 Propagación de Restricciones (Consistencia de Arco)

Consistencia de Arco (AC-3): Un preprocesamiento que "propaga" las consecuencias de las restricciones para reducir los dominios. Si un valor del dominio de la variable X1 no tiene un valor consistente en el dominio de la variable X2 para una restricción entre ellas, ese valor se elimina del dominio de X1.

El algoritmo AC-3 mantiene una cola de arcos y los revisa, eliminando valores y agregando nuevos arcos a la cola si un dominio se reduce. Resultado: la búsqueda posterior es mucho más rápida, y a veces descubre que el CSP es insatisfacible antes de buscar.

5.4 Búsqueda Local para CSPs: Mínimos Conflictos

Mínimos Conflictos: Un algoritmo de búsqueda local muy eficiente para CSPs. Se parte de una asignación completa (aunque sea inválida) y se modifica iterativamente el valor de una variable para reducir el número de restricciones violadas. Es sorprendentemente eficaz para problemas como las n-reinas y la programación de telescopios.